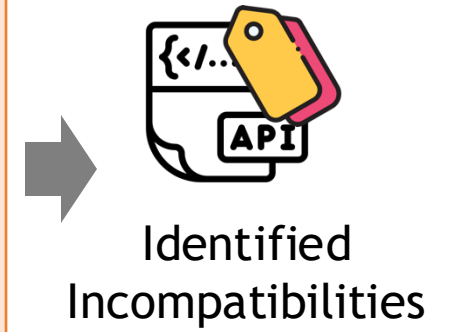
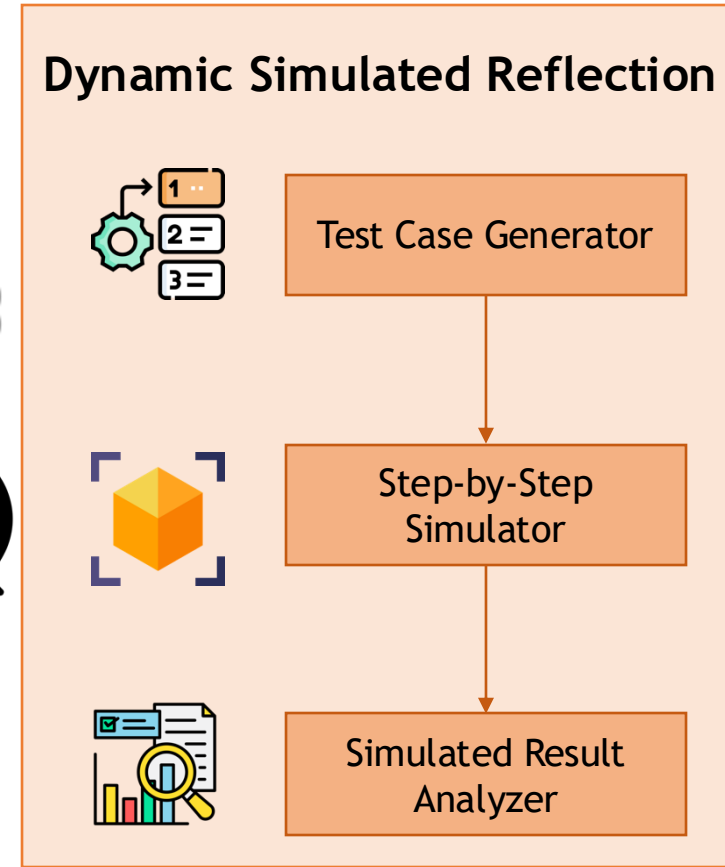
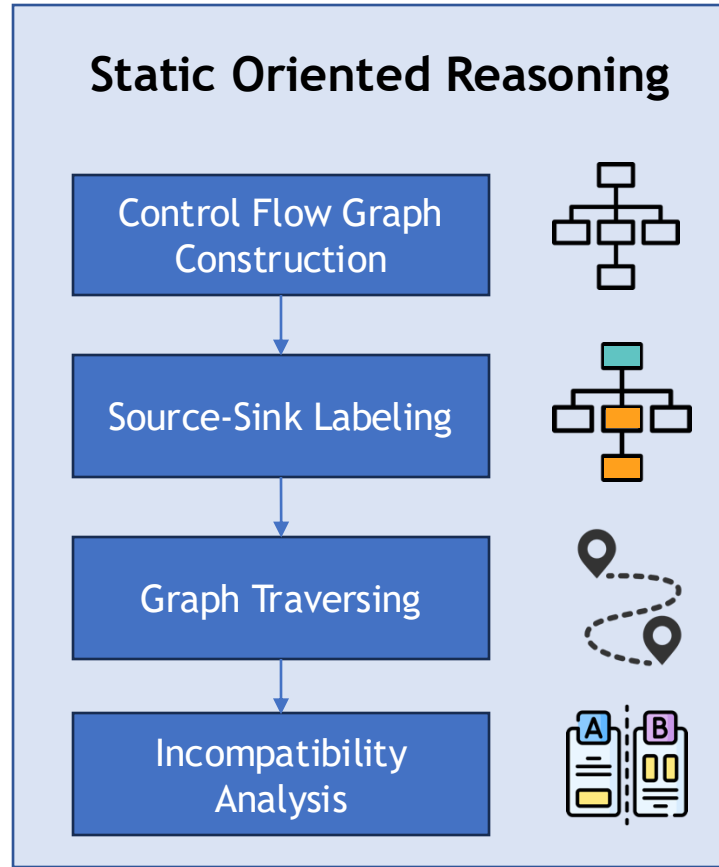
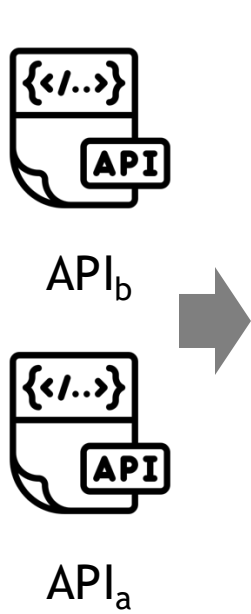




```

1 - def fun(self, amount, n_size = 0, flag = True, **kwargs):
2 + def fun(self, n_dim = 0, is_good = False, **kwargs):
3     global GLOBALX
4 -     self.amount = amount
5     if flag:
6         return True
7 -     GLOBALX = n_size
8 -     new_var = [0] * n_size
9 -     if kwargs.get('key'):
10+    GLOBALX = n_dim if n_dim > 0 else None
11+    new_var = [0] * n_dim
12+    if kwargs.get('key_for_check') and self.type_bad():
13        raise TypeError('message')
14    if len(new_var) == 0:
15-        raise ValueError('msg1')
16+        raise LENError('msg2')
17    return new_var

```

CI1: API_{before} removes required parameter 'amount'. Using old arguments like 'amount=1' triggers "unexpected keyword argument" errors.

CI3: API_{after} renames 'n_size' to 'n_dim'. Using 'n_size=10' in the new API throws "unexpected keyword argument" error

CI4: API_{after} changes 'is_good's default from True to False. Without explicit assignment, the old API returns True, while the new API returns a list, causing behavioral differences.

CI9: API_{after} deletes 'self.amount = amount', so self.amount is unassigned after calling the new API. Accessing obj.amount in upper code triggers AttributeError or gets unexpected values.

CI2: API_{after} changes the TypeError trigger key from 'key' to 'key_for_check'. Only passing 'key=test' may raise a KeyError in new API.

CI6: Code_v2 replaces ValueError with custom LENError when len(new_var)==0. The new API raises different exception

CI7: With the same input (e.g., no explicit is_good), the old API returns True, while the new one returns a list of length n_dim.

CI5: The same input leads to inconsistent exception triggering (e.g., 'key=test' no longer raises TypeError).

